

第 7 讲

DRAM Bandwidth and other Performance Considerations

目录

1 课程安排与核心内容	1
1.1 课程安排与阅读材料	1
1.2 学习目标与知识范围	1
2 DRAM 组织与带宽	2
2.1 从存储单元到二维阵列	2
2.2 选行、选列与突发传输	3
2.3 多个存储体与内存级并行	5
3 Warp 访存与共享内存分块	6
3.1 合并访存的判断单位	6
3.2 向量访问与行主序地址	7
3.3 矩阵乘法：连续访问、广播与跨行步长	9
3.4 共享内存分块：协作加载与数据复用	11
4 线程粒度、指令调度与同步	13
4.1 从细粒度线程到线程粗化	13
4.2 粗化后的地址布局与资源代价	14
4.3 循环展开、指令调度与寄存器使用	16
4.4 双缓冲与同步依赖	18
5 优化权衡与例题	21
5.1 常见优化的功能清单	21
5.2 优化顺序、资源权衡与瓶颈	22
5.3 例题一：线程块形状改变访存优势	24
5.4 例题二：物理带宽与有效输入吞吐率	25

1 课程安排与核心内容

1.1 课程安排与阅读材料

以下课程事项对应 2026 年 9 月 15 日。

Lab 2 提交要求 Lab 2 在授课当周的星期五截止。需要完成全部要求的代码，包括主机内存的分配与释放；将工作推送到 GitHub；代码完成之后再提交 Lab 2 quiz。

课程辅助资源 课程聊天机器人可用于询问设备内存分配、主机到设备的数据复制、GPU 内核 (*kernel*) 的调用、共享内存数组声明，以及向量加法代码的逐行解释。复习时也可要求它围绕 Lecture 2 和 Lecture 3 逐题提问、等待回答并检查正确性。阅读任务是教材第 6 章和 CUDA Best Practices Guide 的 *Memory Optimizations*；待办事项是提交 Lab 2。

1.2 学习目标与知识范围

学习目标 理解动态随机存取存储器的组织方式；理解突发模式和多个存储体如何提高数据传输率；理解连接 GPU kernel 性能与 DRAM 组织的合并访存；掌握多种 kernel 优化手段及其相互权衡。

优化机制之间的关系 每块线程数、每块共享内存量和每线程寄存器数共同影响占用率 (*occupancy*)；减少控制分歧可提高 SIMD 效率，共享内存分块和寄存器分块则减少内存流量。访存与执行优化包括合并访存、面向内存延迟的占用率调节、线程粗化、循环展开和双缓冲。私有化与 *warp* 级原语分别用于减少访问冲突和线程间协调开销。

全局内存的标称带宽与程序实际获得的有效数据率之间可能存在差距。并发访问若争用同一资源，会产生串行化与延迟；为计算核心提供足够多的线程，还需要让这些线程以合适的方式请求数据。

三种互补的改进方向

数据复用减少需要搬运的数据量；合并访存提高必要传输中有效数据的比例；并发请求使内存系统在等待某次访问时仍能处理其他工作。线程粗化、指令调度与双缓冲进一步调整每线程工作量和等待关系。

2 DRAM 组织与带宽

2.1 从存储单元到二维阵列

2.1.1 电容保存比特，晶体管控制连接

随机存取存储器 (*Random Access Memory, RAM*) 允许直接访问任意地址，无须先顺序经过前面的存储位置。动态随机存取存储器 (*Dynamic RAM, DRAM*) 用电容上的电荷保存一个比特，并通过晶体管把该电容接到位线 (*bit line*) 上进行读写。选择信号 (*SELECT*) 决定这个连接是否导通。

电容会漏电，保存的信息因此需要周期性刷新 (*refresh*)；典型时间尺度为数十毫秒。每个比特只需要一个晶体管和一个电容，因此可以在有限面积内布置大量存储单元。

2.1.2 高密度与慢访问来自同一组织方式

在示意组织中，一根位线连接约 1,000 个单元，地址译码选择其中一个单元与位线接通。存储单元的电容很小，长位线的电容却很大；微小电荷改变位线状态需要时间，因此还要使用感应放大器 (*sense amplifier*) 来加速信号检测。

一个 DRAM 存储体 (*bank*) 由二维单元阵列组成：先激活一行，再按列选择数据。一行选择线连接约 1,000 根位线，这与每根位线上的约 1,000 个单元一起形成约 10^6 比特量级的示意阵列。这些数值用于说明组织规模。

各部件的分工 行译码器 (*row decoder*) 根据行地址选择一行；感应放大器检测单元数据；列锁存器 (*column latches*) 保存已取得的行数据；多路选择器 (*multiplexer, Mux*) 再根据列地址选择输出。访问阵列和从已取得的数据中选择输出，构成快慢不同的两个阶段。

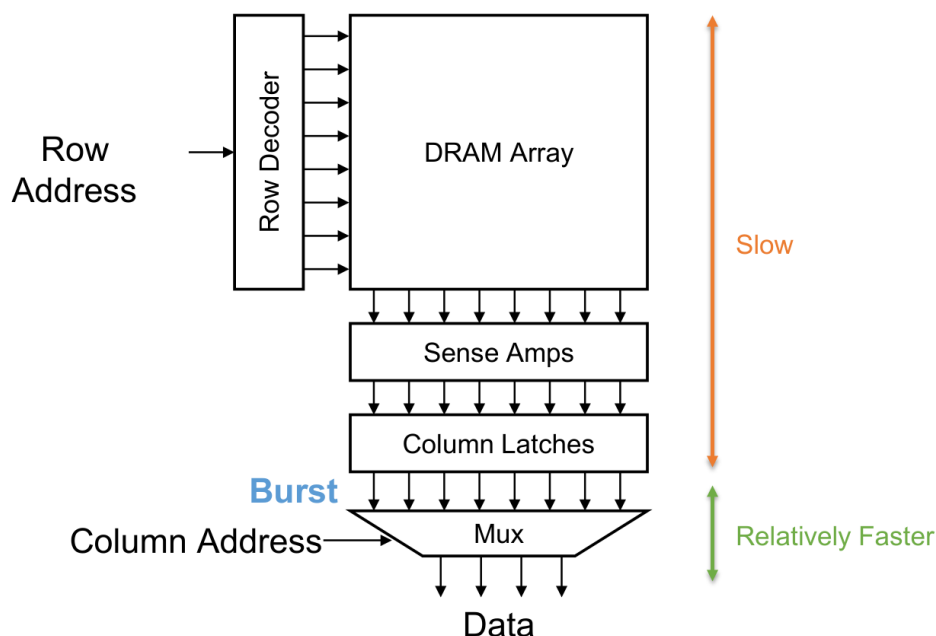


图 1 DRAM bank 的行译码、阵列、感应放大器、列锁存器与多路选择器。上部阵列访问较慢，已取得数据的列选择与输出相对较快。

2.2 选行、选列与突发传输

2.2.1 一次访问的三个阶段

行激活 (*row activation*) 把选定行的数据打开到行缓冲区 (*row buffer*)；列访问 (*column access*) 从已打开的行中选择所需数据；突发传输 (*burst transfer*) 通过内存接口传送一串连续数据。

行地址与列地址承担不同的选择任务。行访问先承担较慢的阵列操作，后续数据可以从已经取得的内容中选出。这样，一次较慢的准备工作可支持多个相邻数据的输出。

2.2.2 8×2 比特小存储体示例

“ 8×2 bit” 表示 8 个可寻址位置、每个位置 2 bit，总计 16 bit。按图中的地址划分，地址的前两位控制行选择，最后一位控制行内数据选择。示意阵列画成 4 行、每行 4 bit。

地址从 010 变为 011 时，仅最后一位改变。两次访问复用已取得的行数据，只改变后续的选择与输出，从而共用较慢的前期工作。

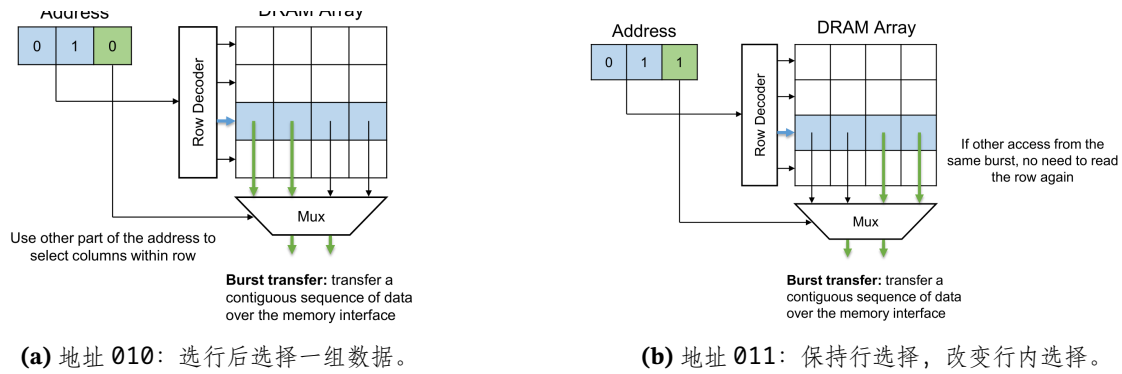


图 2 同一批已取得的数据支持相邻地址的输出。地址前两位以蓝色标记，最后一位以绿色标记。

2.2.3 把慢阶段的成本分摊到多个数据

小存储体的时序模型用橙色表示较慢的阵列访问，用绿色表示较快的数据输出。访问同一 burst 中的数据，可以共用前面的橙色阶段；图中不同 burst 的示意则重复了这个阶段。

示意时序的形式化 设 L 为一次慢阶段的时间， t 为一次快输出的时间， q 为所需输出的数据个数。在“每个数据单独重复慢阶段”和“ q 个数据共用慢阶段”这两个示意情形下，分别有

$$T_{\text{separate}} = q(L + t), \quad (1)$$

$$T_{\text{burst}} = L + qt, \quad (2)$$

$$T_{\text{separate}} - T_{\text{burst}} = (q - 1)L. \quad (3)$$

L 和 t 分别表示模型中的慢阶段与输出阶段耗时；时间式通过比较两阶段的重复次数解释 burst 的收益。

突发传输的带宽含义

连续数据能够分摊一次较慢的准备成本。若一次传输带来的数据大部分都会被使用，接口传输的数据就能更多地转化为程序需要的数据；若只使用其中少量位置，其余传输占用的带宽就没有直接贡献。

2.3 多个存储体与内存级并行

2.3.1 通过重叠操作填补传输间隙

单个 bank 即使使用 burst，两次突发传输之间仍可能存在等待时间。多个 bank 分别处理独立请求时，控制器可以把它们的内部操作重叠起来，使一个 bank 准备数据的时间与另一个 bank 输出数据的时间交错，从而保持数据接口忙碌。

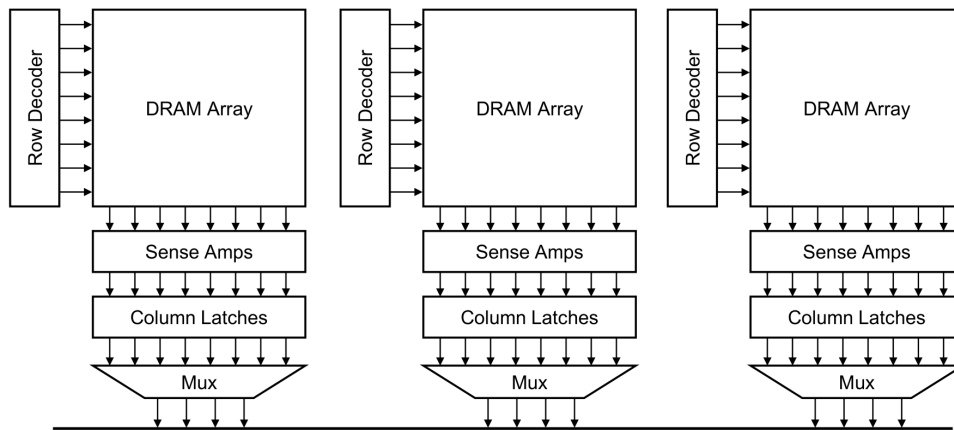


图 3 多个 DRAM bank 具有各自的阵列、译码与数据选择部件，并连接到下方的数据接口。不同 bank 上的独立请求提供可重叠的内部操作。

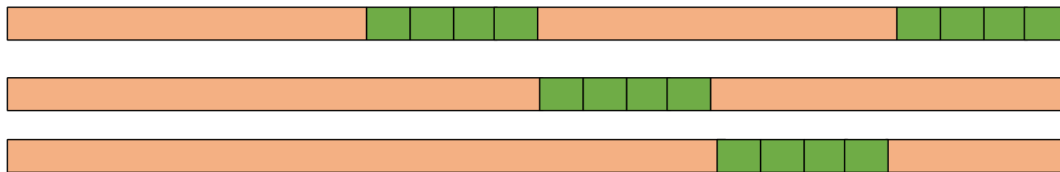


图 4 三个 bank 的慢阶段与突发输出交错排列。绿色输出段依次出现，使整体数据传输能比单个 bank 更连续。

2.3.2 硬件并行需要足够多的独立请求

内存级并行 (*memory-level parallelism, MLP*) 是多个独立内存操作同时处于进行中的状态。多个 bank 提供并行处理的条件，程序则需要持续发出足够多的请求，让这些 bank 有工作可做。

较高的流多处理器 (*Streaming Multiprocessor, SM*) 占用率可以提供更多线程，从而增加可并发的内存访问。这与使用多个线程束 (*warp*) 隐藏计算核心中的流水线延迟具有相同的基本思路：一部分工作正在等待时，仍有其他工作可以推进。

与单次访存效率的关系 合并访存改善一次 warp 访问的数据分布，MLP 改善不同请求在时间上的重叠程度。共享内存分块则通过复用减少需要发出的请求。三者分别作用于传输量、传输效率和请求并发程度。

占用率的权衡 为了提高占用率，需要同时考虑每块线程数、每块共享内存和每线程寄存器。更多驻留线程有助于隐藏延迟，也会共享并竞争这些有限资源；优化目标是提供足够的独立工作，同时保留每个线程或线程块需要的复用空间。

3 Warp 访存与共享内存分块

3.1 合并访存的判断单位

3.1.1 固定一条访存指令，比较同一 warp 的地址

合并访存 (memory coalescing) 是把同一线程束 (warp) 内多个线程对相邻位置的访问组合起来服务。连续访问能够以较少的内存事务 (memory transaction) 传送所需数据；分散到不同位置的请求往往需要更多事务，也会增加 warp 等待数据的时间。这类分散访问也称为**内存分歧 (memory divergence)**。

判断时需要固定同一次访存，再观察 warp 内各线程的地址。例如，在矩阵乘法内积循环中固定 k ，列出线程 $T_0, T_1, \dots$ 此时分别访问哪里。一个线程随 k 变化的访问顺序，与多个线程在固定 k 时的地址分布，是两个不同的观察方向。

访存分析的三个要素

线程块形状决定一个 warp 中哪些线程坐标相邻；地址表达式把线程坐标映射成元素下标；元素类型再把下标差换算成字节距离。确定这三项之后，才能判断同一 warp 的地址是连续、相同，还是带有步长。

3.1.2 二维线程块如何组成 warp

对二维线程块，线程首先沿 x 维连续排列，再进入下一条 y 行。用 $B_x = \text{blockDim.x}$ 、 $B_y = \text{blockDim.y}$ 表示块形状，用 t_x, t_y 表示块内坐标，可将这个排列规则写为

$$t = t_y B_x + t_x, \quad t_x = t \bmod B_x, \quad t_y = \left\lfloor \frac{t}{B_x} \right\rfloor. \quad (4)$$

式中 t 为块内线性线程编号。每 32 个相邻线性线程构成一个 warp；用 $\ell = 0, \dots, 31$ 表示 warp 内的线程槽位 (lane)，某个完整 warp 的线程编号可写成 $t = 32w + \ell$ ，其中 w 为该 warp 在块内的编号。这是对给定排列规则的索引展开。

因此，`dim3(32, 8)` 中的 warp 可以沿完整的一条 x 行排列；`dim3(1, 32)` 的 warp 沿 y 变化；`dim3(16, 16)` 的一个 warp 则由两条各含 16 个线程的 x 行组成。

3.1.3 四种典型地址分布

下面的地址均为字节地址。连续访问中的每个线程读取一个 4 字节元素。

表 1 同一次访存中，不同 lane 的地址关系。

模式	lane 0、1、2、3 等的地址	主要特征
连续合并 <i>coalesced</i>	0, 4, 8, 12, ..., 124	相邻线程请求相邻元素，形成连续的 128 字节需求。
广播 <i>broadcast</i>	40, 40, 40, 40, ...	多个线程请求同一数据，表现为同址复用。
步长访问 <i>strided</i>	0, 16, 32, 48, ...	每次只取相隔 16 字节的位置，地址间留有空隙。
分散访问 <i>scattered</i>	84, 0, 132, 16, ...	地址分散，需逐项检查其覆盖的传输区域。

广播可以满足多个线程对同一数据的需求；连续合并则让多个不同元素一起被有效传输。广播与连续合并对应不同的数据关系，分析复用收益时应分别处理。

在简化模型中，一组连续访问可由一次 burst 或事务服务。更一般地，合并访存减少满足同一 warp 请求所需的事务数量；具体数量还取决于硬件事务粒度与地址对齐。

3.2 向量访问与行主序地址

3.2.1 向量加法：每条指令内的地址都连续

对长度为 n 的单精度向量，线程的全局下标为

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}. \quad (5)$$

核心运算为 $z[i] = x[i] + y[i]$ ，边界判断保证只有有效下标参与计算。

代码 1 向量加法：相邻线程处理相邻元素。

```

1 __global__ void vectorAdd(const float* x, const float* y,
2                           float* z, unsigned int n) {
3     unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
4     if (i < n) {
5         z[i] = x[i] + y[i];
6     }
7 }

```

设一个完整 warp 的第一个全局下标为 i_0 ，则对数组 x 的读取地址为

$$a_{x,\ell} = a_x + 4(i_0 + \ell), \quad \ell = 0, \dots, 31, \quad (6)$$

其中 a_x 是数组起始字节地址。相邻 lane 的地址差是 4B，整个 warp 需要 32 个 float，即 128 B 的连续数据。对 y 的读取与对 z 的写入也分别具有同样的关系。三组数组访问分别分析。

3.2.2 二维 C 数组按行线性存储

行主序 (row-major order) 把同一行的元素连续放入内存，再接下一行。设矩阵列数为 $W = \text{Width}$ ，行列下标分别为 r, c ，则元素下标与字节地址为

$$\text{index}(r, c) = rW + c, \quad a(r, c) = a_0 + s(rW + c), \quad (7)$$

其中 a_0 为矩阵起始地址， s 为每个元素的字节数。固定 r 而令 c 增加 1，地址增加 s ；固定 c 而令 r 增加 1，地址增加 sW 。这是对行主序图的形式化。

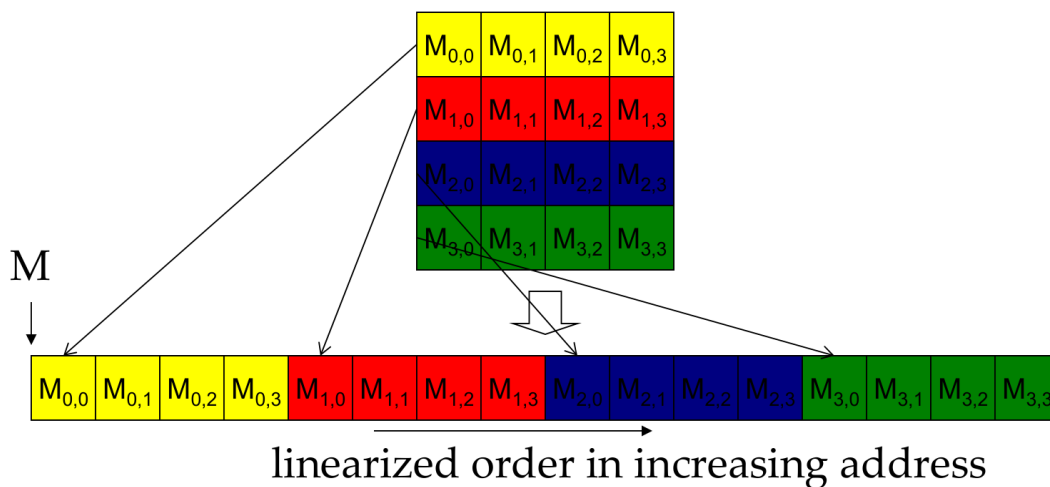


图 5 四行矩阵依次线性存储。每一行以同一颜色表示，内存地址沿下方箭头增大。

矩阵乘法中的 $M[\text{row} \times \text{Width} + k]$ 沿单个线程的 k 循环读取一行, $N[k \times \text{Width} + \text{col}]$ 沿单个线程的 k 循环读取一列。是否合并还取决于同一 warp 内的 row 和 col 如何变化。

3.3 矩阵乘法：连续访问、广播与跨行步长

3.3.1 固定输出元素与固定内积迭代

设 M, N, P 均为 $W \times W$ 矩阵, $P = MN$, 则

$$P_{r,c} = \sum_{k=0}^{W-1} M_{r,k} N_{k,c}. \quad (8)$$

每个线程负责一个输出元素, threadIdx.x 影响输出列, threadIdx.y 影响输出行。下列片段在 kernel 内使用, M, N 为输入指针, Width 为 W ; 假设该线程的行列坐标有效。

代码 2 矩阵乘法的内积访问, Width 表示矩阵维度。

```
1 int row = blockDim.y * blockIdx.y + threadIdx.y;
2 int col = blockDim.x * blockIdx.x + threadIdx.x;
3 float sum = 0.0f;
4 for (int k = 0; k < Width; ++k) {
5     sum += M[row * Width + k] * N[k * Width + col];
6 }
```

3.3.2 沿同一输出行排列的线程

考虑 $\text{blockDim.x}=32$, 一个 warp 位于同一输出行 r , 列下标为 $c_0 + \ell$ 。固定 k 后, 两个输入的元素下标分别为

$$I_M(\ell) = rW + k, \quad (9)$$

$$I_N(\ell) = kW + c_0 + \ell. \quad (10)$$

I_M 与 ℓ 无关, 32 个线程读取同一个 $M_{r,k}$, 对应广播或复用; I_N 随 ℓ 每次增加 1, 线程读取相邻的 N 元素, 对应经典连续合并访问。

沿着单个线程观察, N 的读取会随着 k 增加而跳到下一行; 沿着同一次 k 的 warp 观察, 线程却一起读取 N 的同一行。这解释了“单个线程纵向访问, 但 warp 仍可合并”的现象。

3.3.3 跨输出行排列的线程

若所比较的线程具有 $r = r_0 + \ell$ ，则固定 k 时

$$I_M(\ell) = (r_0 + \ell)W + k, \quad I_M(\ell + 1) - I_M(\ell) = W. \quad (11)$$

相邻线程访问不同行的同一列，相差 W 个元素。单精度数据的字节距离为 $4W$ ，这就是跨行步长访问。

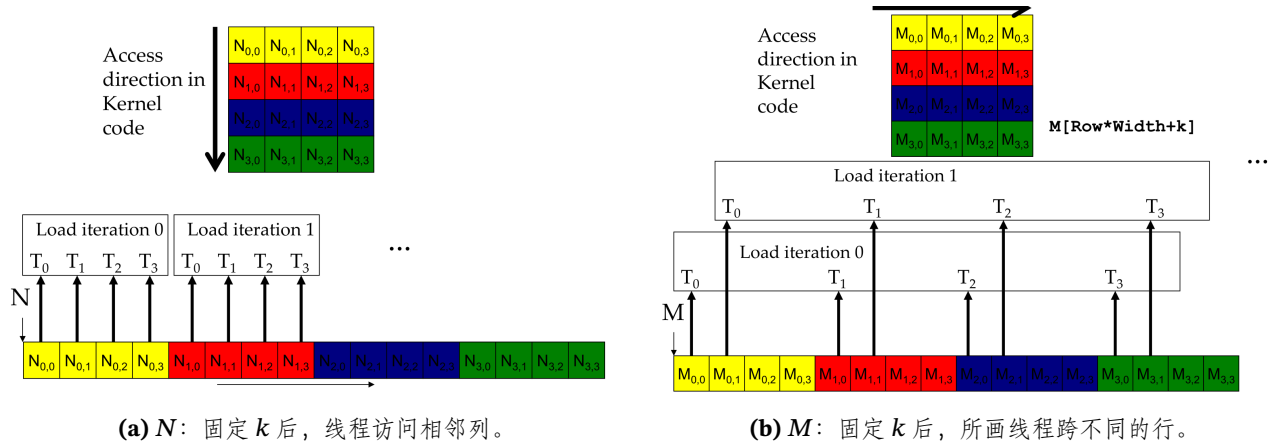


图 6 两个访问方向示意。上方箭头描述单线程沿 k 的移动，下方连线描述固定 k 时线程之间的地址关系。

线程映射决定访问模式 跨输出行的线程读取 M 时形成跨行步长；同一输出行中的线程则对 M 广播。判断访存模式时，需要先明确所比较线程的坐标关系。

块宽为 16 时的索引展开 一个 warp 包含两条输出行、每行 16 个线程。固定 k 时，对 M 的请求形成两组同址访问；对 N 的请求则是同一组 16 个连续列各被访问两次。这可直接由式(4)和内积代码得到，进一步说明应按实际 warp 布局判断。

3.4 共享内存分块：协作加载与数据复用

3.4.1 将数据搬运方式与计算访问方式分开

共享内存分块(*shared-memory tiling*) 让一个线程块协作加载输入子块，再在共享内存中反复使用这些值。全局内存加载按便于合并的顺序分工，乘法运算则按内积所需的行列关系读取共享内存。

设子块边长为 $T = \text{TILE_DIM}$ ，阶段编号为 p ，块内线程坐标为 t_x, t_y 。每个线程在阶段 p 加载

$$M_s[t_y][t_x] \leftarrow M[rW + pT + t_x], \quad (12)$$

$$N_s[t_y][t_x] \leftarrow N[(pT + t_y)W + c]. \quad (13)$$

在同一条块内 y 行上， r 与 t_y 固定， t_x 连续变化， c 也随 t_x 连续变化，因此两个输入的加载地址都连续。

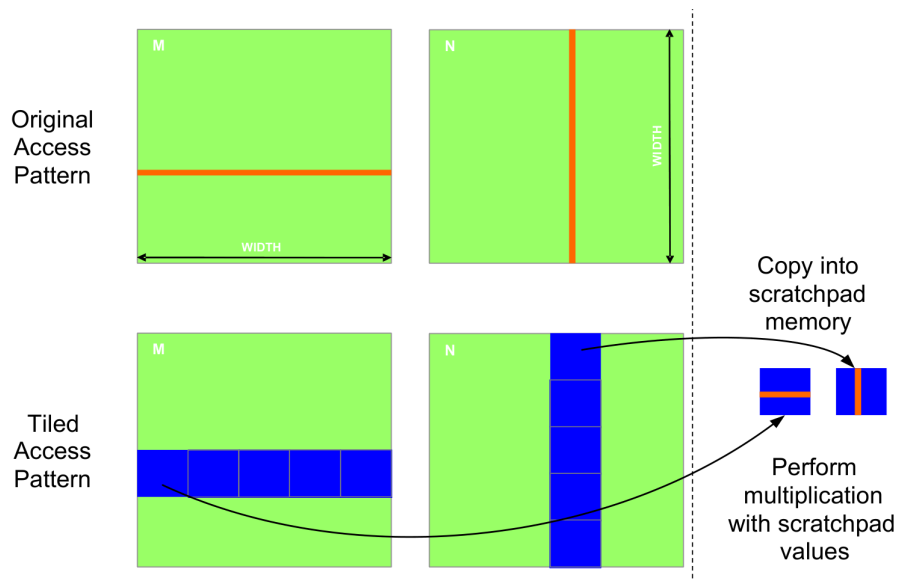


图 7 输入子块先复制到暂存区(*scratchpad memory*)，再使用暂存值执行乘法。蓝色区域表示分阶段加载的子块，橙色线表示内积所需的行列方向。

3.4.2 完整计算阶段与同步位置

方阵分块核函数依次执行协作加载、同步、计算和再次同步。 T 为编译期常量， $Width$ 是 T 的正整数倍；线程块为 $T \times T$ ，网格为 $(Width/T) \times (Width/T)$ 。这些约定使所有加载下标有效。

代码 3 包含协作加载、两次同步和输出写回的分块矩阵乘法。

```

1  template<int T>
2  __global__ void matmulTiled(const float* M, const float* N,
3                             float* P, int Width) {
4      __shared__ float M_s[T][T];
5      __shared__ float N_s[T][T];
6      int tx = threadIdx.x;
7      int ty = threadIdx.y;
8      int row = blockIdx.y * T + ty;
9      int col = blockIdx.x * T + tx;
10     float sum = 0.0f;
11
12     for (int tile = 0; tile < Width / T; ++tile) {
13         M_s[ty][tx] = M[row * Width + tile * T + tx];
14         N_s[ty][tx] = N[(tile * T + ty) * Width + col];
15         __syncthreads();
16
17         for (int k = 0; k < T; ++k) {
18             sum += M_s[ty][k] * N_s[k][tx];
19         }
20         __syncthreads();
21     }
22     P[row * Width + col] = sum;
23 }
```

第一次同步确保两个输入子块已经由所有线程加载完成；第二次同步确保所有线程都用完当前子块，随后才允许下一阶段覆盖这两块共享内存。

3.4.3 合并收益与复用收益分别计算

加载顺序 在 $T = 32$ 的示意下，warp 中的 $t_x = 0, \dots, 31$ ，两个加载式分别请求 32 个连续元素。在简化事务模型中，这类连续加载可由一次事务服务。块宽变化后仍按实际 warp 布局分析，例如 $T = 16$ 时每个 warp 包含两段各 16 个元素的行内加载。

同一输出行直接读取 M 时，整个 warp 取得一个共同元素；协作加载阶段则一次取得一段不同的 M 元素，随后供块内重复使用。这样既提高传输中不同有效数据的利用程度，也保留后续的复用机会。

源代码层面的读取次数 一个 $T \times T$ 输出子块在一个内积阶段需要 T^3 次乘法。逐输出直接读取两项输入的写法，对应 $2T^3$ 次逻辑元素读取；协作加载两个 $T \times T$ 子块，则只在加载阶段执行 $2T^2$ 次全局元素读取。于是

$$\frac{\text{逐输出读取次数}}{\text{协作加载次数}} = \frac{2T^3}{2T^2} = T. \quad (14)$$

这项计数用于解释复用机会：同一输入元素在块内参与多次乘法。广播和缓存会影响实际 DRAM 流量，所以该比值描述代码中的请求计数，不直接等于实际带宽或加速比。

资源代价 代码中两个共享数组共占 $2T^2$ 个 float。按每个 float 为 4 字节，共享内存需求为

$$S_{\text{block}} = 8T^2 B. \quad (15)$$

例如 $T = 16$ 时为 2,048 B， $T = 32$ 时为 8,192 B；对应线程数分别为 256 与 1,024。这是由代码得到的资源计数。更大的子块增加复用机会，同时消耗更多每块资源，因而会与占用率产生权衡。

4 线程粒度、指令调度与同步

4.1 从细粒度线程到线程粗化

4.1.1 细粒度划分与透明可扩展性

细粒度线程划分 (*fine-grain thread granularity*) 把尽可能小的可并行工作单元分给每个线程：向量加法中一个线程处理一个向量元素，RGB 转灰度或模糊操作中一个线程处理一个输出像素，矩阵乘法中一个线程处理一个输出元素。

这种划分向硬件暴露尽可能多的线程。资源充足时，可以同时推进较多工作；线程数超过硬件当时能容纳的数量时，其余工作可由硬件以较低开销分批执行。未来设备具有更多资源时，同一工作划分还能直接提取更多并行性，这就是**透明可扩展性** (*transparent scalability*)。

过细划分的代价 若把工作拆给更多线程会额外引入重复读取、重复计算或同步，这些开销会随着细粒度划分一起出现。只有一部分线程能同时执行时，程序可能已经承担了更多并行化开销，却仍需要等待多批工作依次完成。

4.1.2 一个线程承担多个工作单元

线程粗化 (*thread coarsening*) 把多个可并行工作单元交给同一线程。粗化因子 (*coarsening factor*) 表示每个线程负责的工作单元数。例如，粗化因子为 4 时，每个线程处理四个工作单元。

下列片段在 `kernel` 内使用，`foo(i)` 表示已经定义好的第 i 个独立工作单元的处理函数。未粗化时，每个线程只调用一次：

代码 4 细粒度映射。

```
1 unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
2 foo(i);
```

粗化之后，每个线程顺序处理四个单元：

代码 5 连续四单元分配给同一线程。有效下标范围在该概念片段中作为前提。

```
1 unsigned int iStart =
2     4 * (blockIdx.x * blockDim.x + threadIdx.x);
3 for (unsigned int c = 0; c < 4; ++c) {
4     unsigned int i = iStart + c;
5     foo(i);
6 }
```

粗化把一部分跨线程的工作关系改为线程内部的顺序关系，为复用数据或中间结果、减少重复工作和同步提供机会；具体收益取决于原来为并行化付出了什么开销。

4.2 粗化后的地址布局与资源代价

4.2.1 连续分配给线程可能产生跨线程步长

设一个 `warp` 中相邻线程的全局编号为 $q_0 + \ell$ 。在上面的连续四单元分配中，固定粗化循环下标 c 后，第 ℓ 个 `lane` 的元素下标为

$$i_{\ell,c} = 4(q_0 + \ell) + c, \quad i_{\ell+1,c} - i_{\ell,c} = 4. \quad (16)$$

若 `foo(i)` 访问数组第 i 个 `float`，相邻 `lane` 相隔 4 个元素，即 16 B。每个线程自己的四次访问相邻，但同一次循环迭代中，`warp` 的地址带有步长。

4.2.2 按线程块宽度跨步，保留每轮连续访问

另一种映射让相邻线程在同一轮处理相邻元素，让同一线程在不同轮之间跨过一个线程块宽度。令 $B = \text{blockDim.x}$ ，代码为：

代码 6 保持每次迭代内 warp 连续访问的四倍粗化。

```

1 unsigned int base =
2     blockIdx.x * (4 * blockDim.x) + threadIdx.x;
3 #pragma unroll
4 for (unsigned int c = 0; c < 4; ++c) {
5     unsigned int i = base + c * blockDim.x;
6     if (i < n) {
7         foo(i);
8     }
9 }

```

用 b 表示线程块编号， t 表示块内线程编号，则该映射为

$$i(b, t, c) = 4bB + t + cB, \quad 0 \leq t < B, \quad 0 \leq c < 4. \quad (17)$$

固定 b, c ，相邻线程的下标差为 1；固定 b, t ，同一线程下一轮的下标增加 B 。一个块覆盖从 $4bB$ 到 $4(b+1)B-1$ 的连续 $4B$ 个位置。

例如取 $b = 0, B = 32$ ，一个 warp 的映射如下。表中的数字为元素下标：

表 2 四轮粗化中的线程分工，由式(17)计算。

循环下标 c	lane 0	lane 1	lane 2	lane 31
0	0	1	2	31
1	32	33	34	63
2	64	65	66	95
3	96	97	98	127

因此，对 n 个工作单元，线程块数量应相应改成

$$G = \left\lceil \frac{n}{4B} \right\rceil. \quad (18)$$

最后一个块中， $i < n$ 使超过数组长度的位置不执行处理。

4.2.3 粗化因子与占用率一起选择

粗化可能减少冗余内存访问、冗余计算、同步开销或控制分歧；同时，一个线程也可能需要更多寄存器或其他资源，进而降低占用率。粗化因子过大，还会使可并发线程过少而无法充分利用设备。

相同的粗化因子在不同设备上可能表现不同，因此需要重新调节。影响性能的关系包括：每线程增加了多少有效工作、减少了多少重复开销、每线程资源增加多少，以及仍有多少独立线程或指令可以隐藏延迟。

粗化的两个联合检查

一方面检查工作分配：每轮相邻 lane 是否仍访问相邻元素，网格是否覆盖全部工作；另一方面检查资源代价：增加的线程内工作是否值得以更少的驻留线程来交换。

4.3 循环展开、指令调度与寄存器使用

4.3.1 展开循环体，减少循环控制

循环展开 (*loop unrolling*) 把循环体复制若干次，同时按相同倍数减少循环迭代次数。循环体执行的总工作量保持一致，循环计数、判断和跳转出现的次数则减少。

设 $A(i)$ 、 $B(i)$ 是已定义的操作，原循环为：

代码 7 原始循环执行 16 次，每次处理一个下标。

```
1 for (unsigned int i = 0; i < 16; ++i) {
2     A(i);
3     B(i);
4 }
```

按因子 4 展开之后，每次循环处理四个下标，外层循环只执行 4 次：

代码 8 按因子 4 展开的循环。

```
1 for (unsigned int i = 0; i < 16; i += 4) {
2     A(i);    B(i);
3     A(i + 1); B(i + 1);
4     A(i + 2); B(i + 2);
5     A(i + 3); B(i + 3);
6 }
```

循环展开减少停顿的两条路径是：减少循环分支，以及暴露更多独立指令供后续调度。当缺少分支预测、循环分支具有较长延迟时，减少分支也能减少相应等待。

4.3.2 拉开依赖指令之间的距离

指令调度 (instruction scheduling) 在保持程序依赖关系的条件下重排指令，把相互依赖的指令隔得更远，从而让中间穿插的独立指令填补等待时间。

示例明确假设： $B(i)$ 依赖 $A(i)$ ，而不同 i 之间没有冲突依赖或副作用。也就是每个下标内部存在 $A(i) \rightarrow B(i)$ ，不同下标的操作链相互独立。于是展开后的指令可以排列为：

代码 9 在各迭代独立的前提下先安排 A，再安排依赖它们的 B。

```
1 for (unsigned int i = 0; i < 16; i += 4) {  
2     A(i);  
3     A(i + 1);  
4     A(i + 2);  
5     A(i + 3);  
6     B(i);  
7     B(i + 1);  
8     B(i + 2);  
9     B(i + 3);  
10 }
```

这样， $A(i)$ 与 $B(i)$ 之间插入了另外三个独立的 A 操作。循环展开扩大了可见的指令序列，指令调度再利用其中的独立性。这种单个线程内部可利用的指令并行性称为**指令级并行 (instruction-level parallelism, ILP)**。

若不同迭代之间存在必须保留的依赖，能够重排的范围就由这些依赖限定。例如，示意中的调度依赖于“不同迭代没有冲突”这一明确假设。

4.3.3 编译器指令与编译期常量

循环展开和指令调度通常由编译器自动完成，也可以使用指令控制展开。例如，使用 `##pragma unroll 4`：

代码 10 使用编译器指令指定展开因子。

```
1 #pragma unroll 4  
2 for (unsigned int i = 0; i < 16; ++i) {  
3     A(i);  
4     B(i);  
5 }
```

固定循环范围有利于编译器识别迭代次数。线程粗化常会引入这种短循环；粗化因子为编译期常量时，粗化循环可以完全展开。

4.3.4 常量下标使局部数组更容易提升为寄存器

粗化之后，一个线程可能用小数组保存多个工作单元的状态。例如，从一个标量变为四个元素：

代码 11 具有变量下标的线程局部数组。

```
1 int x[4] = {0, 0, 0, 0};  
2 for (unsigned int c = 0; c < 4; ++c) {  
3     foo(x[c]);  
4 }
```

完全展开后，各次访问的下标成为常量：

代码 12 展开后各次访问具有固定下标。

```
1 int x[4] = {0, 0, 0, 0};  
2 foo(x[0]);  
3 foo(x[1]);  
4 foo(x[2]);  
5 foo(x[3]);
```

固定下标使每个数组元素可分别对应一个寄存器值，从而提供**寄存器提升 (register promotion)** 的机会。变量下标可能使数组需要可寻址的存储，常量下标则便于编译器把各元素分别分配到寄存器；实际分配取决于编译器优化和资源限制。

三种变换的关系 线程粗化改变“一个线程负责多少工作”；循环展开改变“同一线程怎样表达这些工作”；指令调度改变“在依赖允许范围内，先执行哪条指令”。三者可以结合，以减少重复开销、循环控制和依赖等待，同时仍需考虑增加的寄存器需求。

4.4 双缓冲与同步依赖

4.4.1 单缓冲为何需要两次屏障

双缓冲 (double buffering) 用一个缓冲区读取当前数据，用另一个缓冲区写入新数据，消除因共享同一存储位置而产生的**假依赖 (false dependence)**。

设一个线程块内的线程编号为 `tid`, `other` 是该线程要读取的有效块内下标。每轮从其他线程对应的旧状态取得数据, 再更新自己的状态。用已定义函数 `updateValue` 表示状态更新, 单缓冲循环可写为:

代码 13 单缓冲迭代: 读完旧值后才能覆盖, 写完新值后下一轮才能读取。

```
1 for (int j = 0; j < steps; ++j) {  
2     float next = updateValue(buffer[other]);  
3     __syncthreads();  
4     buffer[tid] = next;  
5     __syncthreads();  
6 }
```

第一次屏障: 保护尚未完成的旧值读取 一些线程可能已经读完并准备写入, 而另一些线程仍在读取旧数据。第一次 `__syncthreads()` 要求整个块先完成旧值读取, 随后再覆盖同一数组。它保护的是**写后于读**(*write-after-read, WAR*)的顺序。这个约束源于读写复用了同一存储位置, 因此属于假依赖。

第二次屏障: 确保下一轮读到已完成的新值 下一轮需要使用本轮其他线程产生的新数据, 所以所有写入都要完成, 线程才能进入下一轮读取。这个**读后于写**(*read-after-write, RAW*)的关系来自计算的数据流, 属于**真依赖**(*true dependence*)。

假依赖仍决定单缓冲实现的正确性。改变存储安排、让旧值与新值分别保存之后, 才具备移除第一道屏障的条件。

4.4.2 两个缓冲区交替承担输入与输出

假设 `buffer0`、`buffer1` 是两个大小相同、彼此分离的块内共享缓冲区，初始状态已放入 `buffer0`，并已完成必要的初始化同步。所有参与线程执行相同轮数。上述初始化前提用于明确执行语义，指针交换在代码中显式展开。

代码 14 双缓冲迭代：只保留每轮末尾的屏障。两个指针交换输入与输出缓冲区的角色。

```

1 float* inBuffer = buffer0;
2 float* outBuffer = buffer1;
3 for (int j = 0; j < steps; ++j) {
4     float next = updateValue(inBuffer[other]);
5     outBuffer[tid] = next;
6     __syncthreads();
7
8     float* tmp = inBuffer;
9     inBuffer = outBuffer;
10    outBuffer = tmp;
11 }
```

读取 `inBuffer` 时，其他线程写入的是 `outBuffer`，因此写入新数据不会覆盖本轮仍需读取的旧值。每轮结束的屏障仍然保留，确保所有新数据都已完成写入。交换两个指针后，刚写好的数组成为下一轮输入，原输入数组成为下一轮输出。

表 3 双缓冲的角色随迭代交替，由指针交换代码直接得到。

迭代编号	读取旧状态	写入新状态	交换后的输入指针
$j = 0$	<code>buffer0</code>	<code>buffer1</code>	指向 <code>buffer1</code>
$j = 1$	<code>buffer1</code>	<code>buffer0</code>	指向 <code>buffer0</code>
$j = 2$	<code>buffer0</code>	<code>buffer1</code>	指向 <code>buffer1</code>

循环状态的检查 每次交换之后，`inBuffer` 都指向最新结果，`outBuffer` 指向可供下一轮覆盖的旧缓冲区。交换只改变指针代表的角色，不搬运数组内容。最后一轮也完成交换，因此循环退出后的最新状态仍由 `inBuffer` 指向。以上结论直接来自代码中的赋值顺序。

4.4.3 减少屏障与增加存储之间的交换

按上述两个循环计数，执行 $J = \text{steps}$ 轮时，单缓冲的循环内屏障调用次数为 $2J$ ，双缓冲为 J ；初始化同步不计入这项比较。减少的是用于约束旧值读取与覆盖写入的那一道屏障。

若一个缓冲区占用 S 字节，两块缓冲区需要 $2S$ 字节。在共享内存中实现时，增加的每块存储需求还要与占用率一起考虑。同步开销在总时间中所占比例，以及额外存储对并行执行的影响，共同决定实际收益。

双缓冲保留的数据依赖

旧状态与新状态分别存储，使本轮读取和本轮写入可以不经中间屏障直接衔接；本轮写入与下一轮读取之间仍有真依赖，所以每轮末尾的屏障保留。

5 优化权衡与例题

5.1 常见优化的功能清单

5.1.1 计算利用率

计算利用率优化主要处理可执行工作不足、循环和依赖停顿，以及 SIMD 执行中的空闲线程。

表 4 计算利用率优化的收益与实现方向。

优化	对计算与访存的收益	实现方向
占用率调节	用更多工作隐藏流水线延迟；用更多并行内存请求隐藏 DRAM 延迟。	调节每块线程数、每块共享内存及每线程寄存器等 SM 资源用量。
循环展开	减少循环分支；暴露独立指令；可能促进局部数组进入寄存器而减少内存访问。	利用编译器展开，在适用处使用编译期可知的固定循环范围。
减少控制分歧	提高 SIMD 效率，减少统一执行过程中空闲的计算资源。	重新组织线程与工作、数据的映射，或调整数据布局。

5.1.2 内存利用率

内存利用率优化分别减少无效传输、跨层重复取数和冲突串行化。共享内存分块主要减少全局内存与 SM 之间的重复传输；寄存器分块进一步减少共享内存到寄存器的重复取数。

表 5 内存利用率优化。私有化用于减少共享访问冲突。

优化	对计算与访存的收益	实现方向
合并全局访存	减少等待全局数据的停顿；减少无效流量，提高 burst / 缓存行利用率。	调整线程映射或数据布局；合并地搬入共享内存，再在其中执行所需访问，例如 corner turning 。
共享内存分块	减少全局内存流量及相应等待。	把线程块内重复使用的数据保存在共享内存，减少跨全局内存层的重复传输。
寄存器分块	减少共享内存流量及相应等待。	把线程或 warp 内复用的数据放入寄存器，减少共享内存到寄存器的重复取数。
私有化 <i>privatization</i>	减少原子更新引起的等待、竞争与串行化。	先在私有副本上做局部更新，完成后更新公共副本。

5.1.3 同步延迟与一般工作粒度

Warp 级原语 (*warp-level primitives*) 先在 warp 内完成需要协调的操作，再在块内合并各 warp 的结果，可减少块级屏障和共享内存流量。

双缓冲 把写入和此前的读取放到不同缓冲区，消除写后于读的假依赖，从而删除用于保护该假依赖的屏障。

线程粗化 把多个并行单元分配给同一线程，用于减少不必要的并行化开销。计算和内存两方面的收益均取决于被减少的开销类型。

5.2 优化顺序、资源权衡与瓶颈

5.2.1 从数据搬运到线程内工作

优化目标可以按以下顺序组织。这个顺序先检查传输量，再检查传输效率与并发程度，最后结合每线程工作和实际测量。

1. **减少不必要的数据流量。**通过复用和分块，避免反复搬运已经取得的数据。
2. **提高必要流量的传输效率。**通过合并访存，让一次传输覆盖尽可能多的有效数据。
3. **暴露足够多的并行工作。**通过延迟隐藏和 MLP，使不同请求能够重叠处理。
4. **增加每线程的有效工作。**结合线程粗化和 ILP，减少重复开销并提高独立指令的可见性。

5. 比较优化前后的测量结果。依据应用在目标设备上的表现评价变换。

5.2.2 优化会交换不同资源

占用率、数据复用与每线程工作量共享同一组有限的 SM 资源。增加驻留线程可以隐藏更多延迟，也会增加寄存器、共享内存和缓存的竞争；增加共享内存子块可以提高复用，但可能减少同时驻留的线程块；增加粗化因子可以减少并行化开销，但提高每线程资源需求。

这些关系使单个指标的最大值与实际性能之间存在权衡。合理的配置需要同时保留足够独立的工作，以及支撑每线程有效计算和数据复用的资源。

5.2.3 以应用在目标设备上的瓶颈为依据

瓶颈(bottleneck)是限制应用在某个设备上继续提高性能的约束。它同时取决于应用与设备：相同代码在不同设备上的限制可能不同，同一设备处理不同应用也可能受到不同资源约束。

优化通过增加一种资源的使用，缓解另一种资源的限制。诊断出主要瓶颈之后，才有依据判断应优先减少数据量、改善访存布局、增加并行请求，还是减少同步和线程内停顿。

性能上限与停止条件 将应用性能与对应的**速度极限(speed-of-light performance)**比较，有助于判断还有多大的改进空间，以及何时停止继续优化。比较时需使用相同设备和计算条件下的性能上限。

5.3 例题一：线程块形状改变访存优势

5.3.1 题目与已知信息

下面两个写法使用相同的全局线程坐标。比较它们的全局内存访问效率：

代码 15 两种行列下标映射。A、B 表示数组名；注释中的 A/B 表示待比较的两种写法。

```
1 int tx = blockIdx.x * blockDim.x + threadIdx.x;
2 int ty = blockIdx.y * blockDim.y + threadIdx.y;
3
4 // Form A
5 B[ty * Width + tx] = 2 * A[ty * Width + tx];
6
7 // Form B
8 B[tx * Width + ty] = 2 * A[tx * Width + ty];
```

只给出地址表达式时，答案是尚不能判断，因为线程块的启动形状没有给出。两种写法的输入读取和输出写入都使用各自相同的下标，因此分析该下标在一个 warp 内如何变化即可。

5.3.2 32×8 与 1×32 的相反结果

记 $W = \text{Width}$ 。忽略所有线程共有的起始偏移，只比较相邻 lane 之间的下标变化。

线程块为 $\text{dim3}(32, 8)$ 一个 warp 内的 ty 固定，tx 随 lane 增加 1。写法 A 的下标增加 1，写法 B 的下标增加 W ，因此原例给出的预期优势是 A：

$$I_A(\ell) = I_A(0) + \ell, \quad (19)$$

$$I_B(\ell) = I_B(0) + \ell W. \quad (20)$$

线程块为 $\text{dim3}(1, 32)$ 一个 warp 内的 tx 固定，ty 随 lane 增加 1。此时写法 A 的下标增加 W ，写法 B 的下标增加 1，原例给出的预期优势变为 B：

$$I_A(\ell) = I_A(0) + \ell W, \quad (21)$$

$$I_B(\ell) = I_B(0) + \ell. \quad (22)$$

同一段下标表达式的表现，会随着哪一个线程坐标在 warp 内连续变化而改变。

5.3.3 16×16 : 分别展开两组 16 个线程

考察线程块内第一个 warp，去掉全局起点偏移后，前 16 个 lane 的坐标为 $(t_x, t_y) = (0, 0), \dots, (15, 0)$ ，后 16 个 lane 为 $(0, 1), \dots, (15, 1)$ 。因此两种写法的相对元素下标分别为：

$$I_A : \underbrace{0, 1, \dots, 15}_{t_y=0}, \underbrace{W, W+1, \dots, W+15}_{t_y=1}, \quad (23)$$

$$I_B : \underbrace{0, W, \dots, 15W}_{t_y=0}, \underbrace{1, W+1, \dots, 15W+1}_{t_y=1}. \quad (24)$$

写法 A 形成两段各 16 个元素的行内连续访问；写法 B 在每组 16 个线程内部具有步长 W ，两组之间在对应位置形成相邻的两个元素。具体传输区域的覆盖还要代入矩阵宽度与起点，因此源例要求分别分析两组线程，而不是直接套用“整个 warp 都沿 x 变化”的情形。

表 6 启动形状与 warp 地址分布的对应关系。

线程块形状	warp 内坐标变化	原例的判断
dim3(32, 8)	同一行内 x 连续变化	A 形成连续行访问。
dim3(1, 32)	x 固定, y 连续变化	交换索引后的 B 形成连续访问。
dim3(16, 16)	两条 x 行, 各 16 个线程	对两个半 warp 分别展开地址。

5.4 例题二：物理带宽与有效输入吞吐率

5.4.1 保留题目中的全部假设

每个 float 为 4 字节；输入起点按 512 字节对齐；流式输入数据起初不在缓存中；两次加载所需的每个 512 字节区域总共只读取一次；并发程度足以持续达到 240 GB/s 的物理读取流量；忽略其他流量。求理想的有效输入吞吐率。

代码 16 每个线程使用四个连续元素中的前两个。

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 float temp = A[4 * i] + A[4 * i + 1];
```

512 字节区域与 240 GB/s 均是这个题目的给定条件。有效吞吐率只统计上面两次加载真正使用的输入字节。

5.4.2 先在四元素组内计算有效比例

线程 i 涉及的四元素组为

$$A[4i], A[4i + 1], A[4i + 2], A[4i + 3]. \quad (25)$$

实际只使用前两个元素，因此每 $4 \times 4 = 16 \text{ B}$ 的地址区间中，使用 $2 \times 4 = 8 \text{ B}$ ，另有 8 B 未用于该运算。

表 7 一个 16 字节组中的有效输入位置。

元素	$A[4i]$	$A[4i + 1]$	$A[4i + 2]$	$A[4i + 3]$
相对字节范围	0-3	4-7	8-11	12-15
是否用于 temp	是	是	否	否

5.4.3 再用一个 512 字节区域核对

取 $i = 0, \dots, 31$ 。32 个线程覆盖 32 组、每组 16 字节，共

$$D_{\text{physical}} = 32 \times 16 = 512 \text{ B}. \quad (26)$$

每个线程使用两个 float，对应的有效输入为

$$D_{\text{useful}} = 32 \times 2 \times 4 = 256 \text{ B}. \quad (27)$$

于是有效字节比例为

$$\eta = \frac{D_{\text{useful}}}{D_{\text{physical}}} = \frac{256}{512} = \frac{1}{2}. \quad (28)$$

这里的两次加载共同使用同一个区域，题目规定这个区域只读取一次，所以物理字节数按 512 B 计。

5.4.4 将有效字节比例乘以物理带宽

设 B_{physical} 为物理读取带宽， B_{useful} 为有效输入吞吐率，则在题设条件下

$$B_{\text{useful}} = \eta B_{\text{physical}} = \frac{1}{2} \times 240 \text{ GB/s} = \boxed{120 \text{ GB/s}}. \quad (29)$$

这个结果保留了题目的“理想有效输入吞吐率”口径：物理接口仍按每秒 240 GB 读取，但其中一半字节用于指定的两个输入元素。

两道例题的共同计算方法

先根据实际线程布局展开同一次访问的地址，再对照题目给出的传输区域统计有效字节。第一题强调 **launch** 决定 **warp** 坐标关系；第二题强调所用字节数与所传字节数共同决定有效输入吞吐率。